

2年经验社招微信5面面经

最近才有空整理一下面经，加上HR面一共6面，前前后后大概一个半月的时间

一面（组员面）

算法

1. 洗牌算法：对52张牌洗牌，要求尽量洗乱，而且原牌不能在原位置上重复

```
1 function shuffleArray(array: any[]) {  
2  
3 }
```

供参考的解法：[blog.csdn.net/qq_26399665...](https://blog.csdn.net/qq_26399665)

在leetcode上面也有类似的题目：leetcode-cn.com/problems/shuozuozhe

1. 给定一个字符串，编写一段代码测试该段字符串的括号是否完全闭合。

```
1  
2 // [()]{}{[()()]()} - true  
3  
4 // [(())] - false
```

这道题也算很经典了，leetcode传送门👉 leetcode-cn.com/problems/valid-parentheses

1. 现有新旧两个数组，旧数组经过一系列操作(包括新增、删除、移动等)后可得到新数组。请实现一个函数，找出旧数组上应该执行的操作列表。

```
1  
2  
3 const nodes = ['A', 'B', 'C', 'D'];  
4  
5 const after = ['B', 'A', 'E', 'D', 'C'];  
6  
7 const operations = [];  
8  
9  
10
```

```
11 // 请自行定义VDomElement结构
12
13 function diff(nodes, after) {
14
15
16
17
18 }
19
20
21
```

思想类似于dom diff

1. 数组a[N]，存放了数字1至N-1，其中某个数字重复一次。写一个函数，找出被重复的数字。时间复杂度必须为O(N), 空间复杂度不能是O[N]。函数原型：int find(int a[], int N)
 - a. 有一个二叉树，每个节点的值是一个整数。写一个函数，判断这棵树中是否存在从根到叶子节点的一个路径，这个路径上所有节点之和为某一个值。存在返回1， 否则返回0。

```
1 struct TreeNode
2 {
3     int value;
4     struct TreeNode *left, *right;
5 };
6 int haspath(struct TreeNode *root, int value)
7
```

leetcode传送门👉 leetcode-cn.com/problems/pa...

1. 最长上升子序列 leetcode-cn.com/problems/lo...

面试

1. 问项目相关的问题
2. vue父子组件如何通信
3. 用webpack做过哪些优化（开发时候，打生产包）
4. tree-shaking原理
5. node和浏览器 事件循环的不同之处
6. 讲下vue 为什么要用虚拟dom ,优缺点是？ 性能。
7. dom diff如何比较？
8. 讲下防抖跟节流的原理和应用场景。

9. 讲下你对闭包的理解，以及闭包的应用场景。

10. 讲下http和https的不同之处。

11. 有了解过webassembly吗？

12. Vue父子组件如何通信

参考答案：

在Vue中，父子组件之间的通信主要有以下几种方式：

1. 通过props传递数据：

- 父组件通过 `props` 向子组件传递数据。在父组件模板中使用子组件时，通过属性绑定的方式将数据传递给子组件。

```
1 <ChildComponent :propData="parentData" />
```

2. 在子组件中，通过 `props` 接收数据：

```
1 export default {  
2   props: {  
3     propData: {  
4       type: String,  
5       required: true  
6     }  
7   }  
8 };
```

3. 通过事件发送数据：

- 子组件通过 `$emit` 方法向父组件发送事件，父组件通过监听这些事件来接收数据。

4. 子组件中：

```
1 this.$emit('eventName', data);
```

5. 父组件中：

```
1 <ChildComponent @eventName="handleEvent" />
```

6. 在父组件的 `methods` 中定义 `handleEvent` 方法：

```
1 methods: {  
2   handleEvent(data) {  
3     this.parentData = data;  
4   }  
5 }
```

用webpack做过哪些优化（开发时候，打生产包）

在使用webpack进行开发和打生产包时，可以进行以下优化：

1. 开发环境优化：

- 使用 `webpack-dev-server` 提高开发效率。
- 开启 `source-map` 便于调试。
- 使用 `Hot Module Replacement`（HMR）实现模块热替换，提高开发效率。

```
1 devServer: {  
2   hot: true,  
3   open: true  
4 },  
5 devtool: 'source-map',
```

2. 生产环境优化：

- 压缩JavaScript文件：使用 `TerserPlugin` 压缩和混淆代码。

```
1 const TerserPlugin = require('terser-webpack-plugin');  
2  
3 optimization: {  
4   minimize: true,  
5   minimizer: [new TerserPlugin()],  
6 },
```

- 提取CSS：使用 `MiniCssExtractPlugin` 将CSS提取到单独的文件中。

```
1 const MiniCssExtractPlugin = require('mini-css-extract-plugin');  
2
```

```
3 plugins: [  
4   new MiniCssExtractPlugin({  
5     filename: '[name].[contenthash].css',  
6   }),  
7 ],
```

- 分离代码：使用 `SplitChunksPlugin` 进行代码分离，减少初始加载时间。

```
1 optimization: {  
2   splitChunks: {  
3     chunks: 'all',  
4   },  
5 },
```

- 缓存优化：使用 `[contenthash]` 进行文件命名，利用浏览器缓存。

```
1 output: {  
2   filename: '[name].[contenthash].js',  
3 },
```

Tree-shaking原理

Tree-shaking是通过静态分析代码的模块依赖关系，移除未使用的代码，从而减小打包后的文件大小。它依赖于ES6模块的静态结构特性。基本原理如下：

1. 静态分析依赖：编译时对代码进行静态分析，确定哪些模块和变量被使用。
2. 标记未使用代码：对未被使用的代码进行标记。
3. 移除未使用代码：在生成最终代码时，将标记为未使用的代码移除。
4. Node和浏览器事件循环的不同之处

浏览器和Node.js中的事件循环机制有一些不同：

- 浏览器事件循环：
 - 浏览器的事件循环分为宏任务队列和微任务队列。常见的宏任务包括 `setTimeout`、`setInterval`、UI渲染等，微任务包括 `Promise.then`、`MutationObserver` 等。
 - 事件循环每次执行时，先执行所有的微任务，然后再执行一个宏任务。
- Node.js事件循环：
 - Node.js的事件循环由libuv库实现，分为六个阶段：`timers`、`I/O callbacks`、`idle`、`poll`、`check`、`close callbacks`。
 - 每个阶段有特定的任务队列，在不同阶段处理不同类型的任务。

- Node.js中也有微任务队列，在每个阶段结束后会执行所有的微任务。

Vue为什么要用虚拟DOM，优缺点及性能

虚拟DOM：

- Vue使用虚拟DOM是为了提高性能和跨平台的能力。虚拟DOM是对真实DOM的一层抽象，通过diff算法计算最小的DOM更新量，减少直接操作DOM的次数。

优点：

- 提高性能：减少DOM操作，提高渲染效率。
- 跨平台：通过抽象DOM实现跨平台渲染，如SSR和Weex。

缺点：

- 增加内存开销：维护虚拟DOM需要额外的内存。
- 复杂性：引入虚拟DOM增加了框架的复杂性。

DOM diff如何比较

DOM diff是通过比较新旧虚拟DOM，找出最小的更新操作，以更新真实DOM。Vue的diff算法主要有以下特点：

1. 同层比较：只比较同一层级的节点，不进行跨层级比较。
2. 双端比较：从头到尾、从尾到头进行比较，快速找出相同节点。
3. 标记删除和插入：对新增、删除和移动的节点进行标记，最后统一处理。

防抖和节流的原理和应用场景

防抖（Debounce）：

- 原理：在事件触发后一定时间内不再触发，事件处理函数才会执行。如果在这段时间内再次触发，则重新计时。
- 应用场景：输入框搜索（用户输入完毕后再进行搜索）、窗口大小调整（调整完毕后再处理）。

节流（Throttle）：

- 原理：在事件持续触发时，保证在一定时间内只执行一次事件处理函数。
- 应用场景：滚动事件（滚动过程中每隔一段时间执行一次）、按钮点击（防止多次点击造成多次提交）。

闭包的理解及应用场景

闭包：

- 闭包是指有权访问另一个函数作用域中的变量的函数。当一个函数内部定义了另一个函数时，内部函数会形成一个闭包，能够访问外部函数的变量。

应用场景：

- 实现私有变量：通过闭包实现数据的封装，保护变量不被外部访问。

- 函数柯里化：通过闭包实现函数的柯里化，便于函数复用。

HTTP和HTTPS的不同之处

- HTTP：
 - 超文本传输协议，数据以明文传输，存在安全风险。
 - 使用TCP协议，默认端口80。
- HTTPS：
 - 安全超文本传输协议，使用SSL/TLS协议加密数据传输，保证数据的安全性和完整性。
 - 使用TCP协议，默认端口443。

WebAssembly

WebAssembly (Wasm) 是一种新的二进制格式，旨在提高Web应用的性能。它可以与JavaScript一起运行，主要用于计算密集型任务。

特点：

- 高性能：接近原生速度的性能，适合计算密集型应用。
- 小文件体积：二进制格式比JavaScript文件小，加载速度快。
- 跨平台：支持在不同平台运行，兼容性好。

应用场景：

- 游戏：提高Web游戏的性能和体验。
- 视频处理：提高视频处理和图像处理的性能。
- 科学计算：处理复杂的科学计算和数据分析任务。

以上是针对这些问题的详细解答，如果有需要进一步了解的地方，可以提供具体的案例或问题，我会进一步解释。

二面（组长面）

笔试：

LRU算法 leetcode传送门👉 [leetcode-cn.com/problems/lr...](https://leetcode-cn.com/problems/lru-cache/)

面试：

1. 项目相关
2. 浏览器事件循环
3. hybrid开发js与原生交互的原理

虽然问的问题不多，但是每个问题问的都挺深

三面（总监面）

算法：

三数之和

给定一个包含 n 个整数的数组 `nums`，判断 `nums` 中是否存在三个元素 a, b, c ，使得 $a + b + c = 0$ ？找出所有满足条件且不重复的三元组。

注意：答案中不可以包含重复的三元组。

例如，给定数组 `nums = [-1, 0, 1, 2, -1, -4]`，

满足要求的三元组集合为：

```
1
2  [    [-1, 0, 1],
3
4      [-1, -1, 2]
5
6  ]
```

概率题：

某城市有两种颜色的出租车：蓝色和绿色（市场占有比例为15：85）。一辆出租车夜间肇事后逃逸，但还好当时有一位目击证人，这位目击者认定肇事的出租车是蓝色的。但是，他“目击的可信度”如何呢？公安人员在相同环境下对该目击者进行“蓝绿”测试得到：80%的情况下识别正确，20%的情况不正确。请问可以算出在有目击证人情况下肇事车辆是蓝色的概率为多少？

面试：

1. 项目相关问题,一直深挖

四面（交叉面）

算法：

1. 设定关系（ $A=1, B=2, C=3, \dots, Z=26, AA=27, AB=28, \dots, AAA=XXX, \dots$ ），写一个转换函数，根据上面规则把一个字符串转换为数字。


```
const convert = (str) =>{
  let res = 0;
  const strArr = str.split('');
  let j;

  const { length } = strArr;
  for(let i = length - 1, j = 1; i >= 0; i--, j *= 26){
    const value = strArr[i].toUpperCase();
    if( value < 'A' || value > 'Z'){
      return 0;
    }
    //value.charCodeAt(0) - 64可以算出A=1,B=2,C=3等对应关系
    res = res + (value.charCodeAt(0) - 64) * j
  }
  return res
}
```

1. 二叉搜索树的第k大节点 [leetcode-cn.com/problems/er...](https://leetcode-cn.com/problems/er-cha-sou-suo-shu-de-di-k-da-jie-dian/)

题目的附加条件是不能二叉树全量存储到另外的存储空间。

所以不能直接中序遍历再去取数组的倒数第三个元素。

```

const kthLargest = (root,k) => {

    let kthValue = 0;

    const helper = (root) => {
        if(!root){
            return;
        }
        helper(root.right);
        if(!--k){
            kthValue = root.value;
            return;
        }
        helper(root.left);
    }
}

```

1. 从一个整数数组中，找出位置连续的最长递增序列
2. 单链表中间位置的节点

```

//双指针，快指针每次走两步，慢指针每次走一步
const middleNode = (head) =>{
    let slow = head;
    let fast = head;

    while(fast && slow){
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}

```

1. 两个有序（从小到大）单链表{value:Number;next:Object}，找出这两条链表中value相等的节点，构造成一条新的链表。

```

const commonNodeList = (l1,l2) => {
  const newList = new ListNode(0)
  //哪个l的value比较小就往next移
  while(l1 && l2){
    if(l1.value === l2.value){
      newList.next = l1.value;
      l1 = l1.next;
      l2 = l2.next;
    }else if(l1.value < l2.value){
      l1 = l1.next;
    }else{
      l2 = l2.next;
    }
  }
  return newList
}

```

1. 反转数组，在原数组反转（不使用arr.reverse方法）

```

const ownReverse = (arr) =>{
  const { length } = arr;
  for(let i = 0, j = length - 1; i < j; i++,j--){
    [arr[i],arr[j]] = [arr[j],arr[i]]
  }
  return arr
}

```

面试:

项目相关

延伸出H5如何调用原生方法

在项目中使用的性能优化相关的手段

五面（GM面）

面试:

项目相关（成长最快的一段时间以及原因，项目迭代周期，重构原因，测试周主要工作，自动化测试）

也问了H5如何调用原生方法

开放性问题：

假设现在有一个web版的微信，使用扫码进行登录的原理，延伸到除了轮询还有什么其他方式，http1.1通过哪个字段可以设置长链接

假设微信好友上限是10000，那如何保证好友列表渲染的性能

对于好友头像（大量小图片的加载）有什么优化手段

在好友列表中，还需要有个好友搜索的功能，模糊搜索，如何在不用所有好友进行一次遍历的情况下找到匹配的人名？（哈希表？）

大文件如何实现断点续传？

如何实现秒传？（相同的文件不需要重新再发一次），延伸至如何判断文件内容是否相同？

每轮面试都会遇到与算法相关的题目，所以算法非常之重要，还有项目中遇到的最大挑战，还有从项目中延伸的跟基础相关的一些问题，项目的话也很注重深度，所以项目和算法都是需要去好好准备的，其他的前端基础相关的也挺多考察的，也蛮注重知识深度的，会一直深挖背后的一些问题。